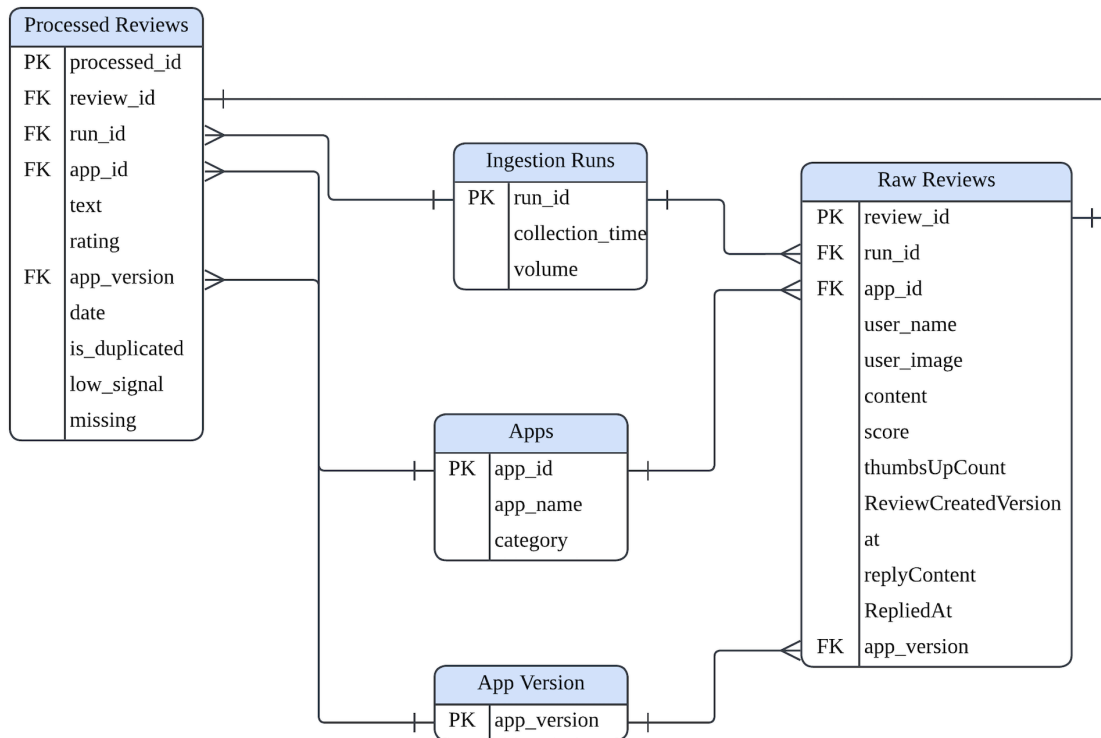




I got 5 tables shown above and have marked primary key and foreign key in the table.

Ingestion Runs: we only need run_id to uniquely identify which ingestion run the run belongs to

Apps: Same as above, we only need app_id to uniquely identify each app

App Version: the primary key is app_version itself.

Raw Reviews: this table contains all information that can be scraped through the Google Play Store Scraper and also includes a primary key, review_id, to uniquely identify each review, while also including 3 foreign keys to connect the three tables above.

Processed Reviews: this table contains less information than the Raw Reviews table do, but its information is more meaningful and is cleaned. The primary key is processed_id, and I also include 3 foreign keys to connect the three basic tables.

All relationships are one-to-many relationships, except the relationship between Processed Reviews and Raw Reviews because one raw review corresponds to only one processed review. For one-to-many relationship, we got the following logic:

- Every ingestion run can generate multiple raw reviews, so is the processed reviews table
- Every app can have multiple raw reviews, and then processed reviews
- Every app version can be related to many raw reviews, so is the processed reviews

Deduplication Logic

In order to prevent duplication, we can use `run_id`, `app_id` and `review_id` together to see if a user's reviews appear more than once for the app and it is scrapped for more than once. Also, in the Processed Reviews table, which is used for later analysis, I include fields, such as `is_duplicated`, `low_signal`, `missing`, these are boolean column to indicate whether it is duplicated, low signal and have missing values. This design gives more direct information about the entire review dataset. If a problem exists, we can quickly find out and deal with it.

Tradeoffs

1. I separate the raw reviews and processed reviews into two tables so we can always access the raw dataset if we encounter any problems with the processed dataset. Even though it takes more space to store, I think reproducibility or traceability is more important than storage.
2. I incorporate quality flags in the processed reviews table instead of in a separate table because it enables us to see clearly what the problem is with the specific review.
3. Even though it might increase query complexity to relate two reviews tables many times with the three tables in the middle, this provides a more clear and maintainable database structure (less repeated rows and keeps the schema lightweight and extensible with a great number of categories)